AttackLab 实验报告

姓名：_____ 学号：_____ 日期：_____

阶段一 dark-calc 的 input 栈帧与长度实验

本阶段分析 attack/dark-calc 的 input 函数，还原它的栈帧布局与各寄存器的取值，并用不同长度的输入记录写入范围与终止方式。

1 任务范围与验证环境

本阶段对应 README 中的三项：观察 input 栈帧、用不同长度输入测试 dark-calc、用 GDB 检查 main 的机器码。

被分析的二进制是 attack/dark-calc，代码段位于固定地址 0x400000 起，因此 0x401977、0x401ad1 这类代码地址每次运行都一样；栈地址以 0x7 开头，受 ASLR 影响每次不同，报告里只比较内容与相对偏移。

查看反汇编所用的命令与工作笔记一致，先在仓库根目录启动 GDB：

```
gdb ./attack/dark-calc
```

进入 GDB 后：

```
set pagination off
disassemble input
disassemble main
```

三段反汇编各自导出成文件：artifacts/01_input.S、artifacts/01_eval.S、artifacts/01_main.S。input 需要重新导出时用笔记里的那一条：

```
objdump -d attack/dark-calc | sed -n "/<input>:/,/^$/p"
```

eval 与 main 用同样的方式，把函数名换掉即可。运行时观察所用的 GDB 命令针对长度实验，放在第 6 节说明。

2 输入格式与判定位置

程序启动后进入 while(1) 循环，每一轮先打印提示语，再从标准输入读一行，交给 input 处理，随后由 eval 拼接命令并执行。

输入是一条以换行为结束的文本行，gets 丢弃换行符并补上结束零字节。判定发生在两处，都在 input 内部：

位置	指令	判定内容	判定成立后的走向
input+40	test %rax,%rax	gets 是否返回 NULL，即读入是否失败	跳到 input+56，打印答案后 exit(0)
input+52	test %al,%al	读到的首字符是否为结束零字节，即是否空行	不跳，落到 input+56

两处判定合起来的效果：读入失败或空行都会打印 answer 并直接退出整个进程。

涉及的三个常量地址：

地址	内容	用途
0x401ce8	Enter an expression (length up to %d):	printf 的格式串
0x401cd0	42_is_The_Answer	空输入或 EOF 时打印
0x6030d0	command_buf，0x1f 字节	后半段的命令缓冲区

程序期望的输入是 15 个字符以内的表达式，交给 eval 拼接成命令后由 shell 执行。超过 15 个字符不会被拒绝，写入会越过数组边界，后果见第 5 节与第 7 节。

3 关键数据结构

局部数组在 input 的帧内，从 rbp-0x10 开始，共 16 字节：

```
#define MAXLEN 15
char expression[MAXLEN + 1];
```

全局数组在数据段，长度 31 字节，取自 01_input_annotated.S 里标注的 nm -S attack/dark-calc | grep -i command_buf，输出为 0x1f：

```
char command_buf[MAXLEN + 16];
```

command_buf比第一个数组大15字节，16个字节能装下15个字符，31个字节能装下30个字符。

4 逐个函数的分析

4.1 input input 的完整反汇编见 artifacts/01_input.S，逐条注释见 artifacts/01_input_annotated.S。按功能分成五段：

段落	指令范围	作用
序言	input+0 到 input+7	保存调用者的 rbp，建立本帧基址，为数组留出 16 字节
提示	input+8 到 input+27	打印提示语，%d 填入 0xf
读取	input+28 到 input+39	把 rbp-0x10 交给 gets
判定	input+40 到 input+75	处理 EOF 与空行，或转入复制
复制与收尾	input+76 到 input+95	strcpy 到 command_buf，然后 leave、ret

关键的一条是 input+88 的 call strcpy@plt：源地址为 rbp-0x10，目的地址为 0x6030d0，复制的长度由输入长度决定。这一条把越界后的字节一并带进全局数组。

图 1 是进入函数体之后的相对位置关系。地址取自一次实测运行，rbp 为 0x...fe0，数组从 0x...fd0 起，返回地址在 0x...fe8；实际数值随 ASLR 变化，能跨运行比较的是相对偏移。

```
高地址
 0x721000801000  +---------------------------+
                 |  上一层帧的内容           |
 0x721000800ff0  +---------------------------+
                 |  返回地址 0x401ad1        |  8 字节
 0x721000800fe8  +---------------------------+
                 |  保存的 rbp 0x721000800ff0 |  8 字节
 0x721000800fe0  +---------------------------+  <- rbp 指向这里
                 |  expression[0..15]        |  16 字节
                 |                           |
 0x721000800fd0  +---------------------------+  <- rsp
低地址
```

图 1 input 进入函数体之后的栈布局。expression 占 16 字节，紧随其上是保存的 rbp，再往上是返回地址。

4.2 eval eval 先截断，再拼接，完整反汇编见 artifacts/01_eval.S：

```
int len = strlen(command_buf) > MAXLEN ? MAXLEN : strlen(command_buf);
memmove(command_buf + 8, command_buf, len+1);
memcpy(command_buf, wrapper_front, 8);
strcpy(command_buf + 8 + len, wrapper_back);
```

三步分别对应 0x401a1b（memmove）、0x401a2f（memcpy）、0x401a43 到 0x401a54（写入右括号）。右括号写在 command_buf + 8 + len 处，于是第 len+1 个字符的位置被覆盖。用 16 个 1 验证过：0x401a57 停下时 command_buf 是 echo $((加 15 个 1 加))，共 25 个字符，第 16 个 1 被右括号覆盖。

截断发生在 strcpy(command_buf + 8 + len, wrapper_back); 这一行。它从 8 + len 处开始写))，len 达到 15 时正好落在第 16 个字符的位置，把多出来的字符挤掉；送入 system 的串因此只有 15 个字符的表达式。

4.3 main 与机器码检查 main 的反汇编如下（任务 5 的对象），完整导出见 artifacts/01_main.S:

```
0x401ac3 <+0>:  push   %rbp
0x401ac4 <+1>:  mov    %rsp,%rbp
0x401ac7 <+4>:  mov    $0x0,%eax
0x401acc <+9>:  call   0x401977 <input>
0x401ad1 <+14>: mov    $0x0,%eax
0x401ad6 <+19>: call   0x4019d7 <eval>
0x401adb <+24>: jmp    0x401ac7 <main+4>
```

从这段机器码能读出:

一是 main 只有序言没有尾声，push 与 mov 之后直接进入循环，结尾是 jmp 回到 main+4，没有 leave 也没有 ret。循环靠 exit 或崩溃退出。

二是 0x401ad1 是 call input 的下一条指令，它同时就是 input 的返回地址，rbp+8 那格存放的值。

三是两次 call 之前都执行 mov $0x0,%eax，这是可变参数调用前清零 al 的习惯写法。

三点里最要紧的是第二点，它把 main 的循环、input 的返回地址和后面的长度实验连在一起。

0x401ad1 是 call input 后面那条指令。call 执行时会把下一条指令的地址压到栈上，因此 input 一进入函数，rbp+8 那格存放的就是 0x401ad1；GDB 里 x/gx $rbp+8 读到的正是这个值，info frame 也把它列为返回地址。input 的 ret 把这一格弹进指令指针，程序就从 0x401ad1 继续执行，rsp 同时回到 0x...ff0，与 rbp 指向同一个位置。

这 8 个字节决定控制流的去处。输入长度到 24 时，结束零字节落在 rbp+8，最低字节从 d1 变成 00，地址变为 0x401a00；长度到 100 时，整格被 0x31 填满，ret 取到的是一个没有映射的地址。0x401a00 落在 eval 的中段，说明改写一个字节就足以把控制流送进另一个函数内部，这也是长度实验中 135 与 139 两个不同信号的来源。

5 写入长度与越界边界

写入的字节数由输入长度决定，与数组容量无关:

写入字节数 = 输入字符数 + 1（那 1 个是 gets 补的结束零字节）

按这个式子推算，各个长度落在哪里:

输入字符数	写入字节数	结束零字节落点	对应位置	实测或推算
3	4	rbp-0xd	数组内部	实测
15	16	rbp-0x1	数组最后一字节	实测
16	17	rbp+0	保存的 rbp 最低字节	实测
24	25	rbp+8	返回地址最低字节	实测
31	32	rbp+0xf	返回地址最高字节	推算
100	101	远超整帧	未映射区域方向	实测

另有一条与显示有关的计算关系，来自 eval：真正送进 system 的算式里，表达式部分最多 15 个字符，命令总长最多 25 个字符（8 个前缀、15 个字符、2 个右括号）。

这两条关系管的是不同的事。gets 按输入长度写栈，写入字节数等于输入字符数加 1，第 16 个字符写到数组最后一字节，第 17 个字节落到保存的 rbp 上，第 25 个字节落到返回地址上，数组的 16 字节容量只决定越界从哪一字节开始。eval 里的 15 与 25 管的是送进 shell 的命令串长度，输入 16 个字符时，栈上写入了 17 字节，命令串里只剩 15 个字符，两个数字由不同的代码决定，测试时可以把它们分开记录。

6 GDB 验证记录

字节级的观察按下面的顺序进行。先停到 input+35：

```
break *input+35
run
```

为了让每次的输入固定，改用已生成的输入文件重定向，执行 ni 时不必再手工键入：

```
run < artifacts/length_input.txt
```

停住后先记录写入前的状态，执行一条 ni 走完 gets，再记录一次：

```
set $buf = $rdi
set $frame_base = $rbp
x/32xb $buf
ni
x/32xb $buf
x/2gx $frame_base
```

这些命令只读取寄存器与内存，不修改程序状态。

下面三张表分别是序言、leave 与 ret、区域定位的读数，均在交互式会话中采集。运行时代码地址固定，栈地址每次不同。

序言三条指令的寄存器变化（停在 input 入口，连续三次 ni）：

时刻	下一条指令	rsp	rbp
断点处	push %rbp	0x...fe8	0x...ff0
执行 push 后	mov %rsp,%rbp	0x...fe0	0x...ff0
执行 mov 后	sub $0x10,%rsp	0x...fe0	0x...fe0
执行 sub 后	mov $0xf,%esi	0x...fd0	0x...fe0

leave 与 ret（停在 input+94，两次 ni）：

时刻	rsp	rbp	下一条指令
leave 前	0x...fd0	0x...fe0	0x4019d5
leave 后、ret 前	0x...fe8	0x...ff0	0x4019d6
ret 后	0x...ff0	0x...ff0	0x401ad1

帧内三块区域的定位（停在 input+94，x/32xb 与 x/2gx）：

区域	相对 rbp	字节数	依据
expression	rbp-0x10	16	sub $0x10,%rsp，+28、+32 两条把该地址送进 rdi
保存的调用者 rbp	rbp+0	8	push %rbp，读出 main 的帧基址
返回地址	rbp+8	8	call 压栈，值为 0x401ad1 即 main+14

断点位置的选择理由：input+35 之前 lea 与 mov 已经执行完，rdi 里就是缓冲区地址，gets 尚未调用，执行一条 ni 即可拿到写入完成的状态；input+94 是 leave，此时整帧数据完整，尚未开始拆解。

最有说服力的一次是 24 个 1 的写入前后对照。x/32xb $buf 在 ni 前后各取一次，x/2gx $frame_base 同时读出 rbp 那格与返回地址那格，三组数字落在同一张表里：数组 16 字节由全零变为 16 个 0x31，rbp 起 8 字节由 f0 0f 80 00 10 72 00 00 变为 8 个 0x31，rbp+8 首字节由 d1 变为 00。这一次观察同时给出了越界起点、覆盖内容与零字节落点，比单看崩溃与否更能说明问题。

7 验证结果

长度实验的实测结果如下，程序在第二次读取遇到 EOF 后打印答案并 `exit(0)`：

输入字符数	退出码	现象
3	0	正常，打印算式与结果后退出
15	0	正常，数组刚好填满
16	0	越界 1 字节，落在保存的 `rbp` 上，退出码仍为 0
24	135	`SIGBUS`，返回地址被改为 `0x401a00`
100	139	`SIGSEGV`，跳到无映射地址

越界写入的字节证据（24 个 `1`）：

地址	写入前	写入后	解释
`rbp-0x10` 到 `rbp-0x1`	未初始化	16 个 `0x31`	字符填满数组
`rbp+0` 到 `rbp+7`	`f0 0f 80 00 10 72 00 00`	8 个 `0x31`	字符越过边界，覆盖保存的 `rbp`
`rbp+8`	`d1`	`00`	结束零字节覆盖返回地址最低字节

第一条关于越界位置。写入字节数等于输入字符数加 1，数组只有 16 字节，第 17 个字节起就写到数组之外。24 个 `1` 时，`rbp` 起 8 字节被 `0x31` 填满，`rbp+8` 首字节被零字节改成 `00`，越界的终点落在返回地址的第一字节上。

第二条关于 16 为什么不崩。零字节只改到保存的 `rbp` 的最低字节，`0x...ff0` 变成 `0x...f00`，另外 7 个字节保持原样。`main` 里没有 `leave` 与 `ret`，整格 `rbp` 不会被用来拆帧；唯一用到它的是 `eval` 里的 `mov %eax,-0x4(%rbp)`，写入落在仍然映射的栈上。第二次读取遇到 EOF，程序打印答案后 `exit(0)`，退出码因此是 0。

第三条关于 24 与 100 的信号差异。24 只改返回地址的最低字节，`0x401ad1` 变为 `0x401a00`，这个地址仍在代码段内，`ret` 取到的是合法地址，随后 `0x401a00` 那条指令以 `0x3131313131313131` 作为帧基址访问内存，触发 `SIGBUS`。100 把整格写成 `0x3131313131313131`，`ret` 直接把没有映射的地址送入指令指针，触发 `SIGSEGV`。`SIGBUS` 具体发生在哪一条指令尚未单步验证，这一点在报告中标为推测。

8 运行截图

五张截图依次如下，所用命令与断点与第 6 节一致。

```
~/Lab3-AttackLab main ?3 ❯ gdb -batch -q -ex 'set pagination off' -ex 'set confirm off' \
  -ex 'break *input+35' -ex 'run < /tmp/n3.txt' \
  -ex 'set $buf=$rdi' -ex 'set $frame=$rbp' -ex 'ni' \
  -ex 'printf "buf=%p  rbp=%p  rsp=%p\n", $buf, $frame, $rsp' \
  -ex 'x/16xb $buf' -ex 'x/2gx $frame' ./attack/dark-calc
Breakpoint 1 at 0x40199a
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
Enter an expression (length up to 15):

Breakpoint 1, 0x000000000040199a in input ()
0x000000000040199f in input ()
buf=0x721000800fd0  rbp=0x721000800fe0  rsp=0x721000800fd0
0x721000800fd0: 0x31    0x31    0x31    0x00    0x00    0x00    0x00    0x00
0x721000800fd8: 0x00    0x00    0x00    0x00    0x00    0x00    0x00    0x00
0x721000800fe0: 0x0000721000800ff0      0x0000000000401ad1
```

图 2 栈帧内的三块区域。3 个 `1` 的输入在 `input+35` 停住并执行一次 `ni` 之后的状态。`buf` 与 `rsp` 同为 `0x...fd0`，`rbp` 为 `0x...fe0`；`x/16xb` 显示数组内只有前 4 字节被写入，`x/2gx` 读出保存的 `rbp` 为 `0x...ff0`、返回地址为 `0x401ad1`。

```
~/Lab3-AttackLab main ?3 ❯ gdb -batch -q -ex 'set pagination off' -ex 'set confirm off' \
  -ex 'break *input+94' -ex 'run < /tmp/n3.txt' \
  -ex 'printf "[leave 前]\n"' -ex 'info registers rsp rbp' -ex 'ni' \
  -ex 'printf "[leave 后]\n"' -ex 'info registers rsp rbp' -ex 'ni' \
  -ex 'printf "[ret 后]\n"' -ex 'info registers rsp rbp' -ex 'x/i $pc' ./attack/dark-calc
Breakpoint 1 at 0x4019d5
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
Enter an expression (length up to 15):

Breakpoint 1, 0x00000000004019d5 in input ()
[leave 前]
rsp            0x721000800fd0      0x721000800fd0
rbp            0x721000800fe0      0x721000800fe0
0x00000000004019d6 in input ()
[leave 后]
rsp            0x721000800fe8      0x721000800fe8
rbp            0x721000800ff0      0x721000800ff0
0x0000000000401ad1 in main ()
[ret 后]
rsp            0x721000800ff0      0x721000800ff0
rbp            0x721000800ff0      0x721000800ff0
=> 0x401ad1 <main+14>:  mov    $0x0,%eax
```

图 3 leave 与 ret 的寄存器变化。同一输入下 input+94 的三次读数。leave 前 rsp 为 0x...fd0、rbp 为 0x...fe0；leave 后 rsp 升到 0x...fe8，rbp 恢复为 0x...ff0；ret 后 rsp 到 0x...ff0，$pc 指向 0x401ad1，即 main+14。

```
~/Lab3-AttackLab main ?3 ❯ ./attack/dark-calc < /tmp/n16.txt; echo "exit=$?"
Enter an expression (length up to 15):
Executing: echo $((111111111111111))
111111111111111
Enter an expression (length up to 15):
42_is_The_Answer
exit=0
```

图 4 长度 16 的终端输出。16 个 1 的输出。Executing 那行只有 15 个 1，第二次读取遇到 EOF 后打印答案，退出码为 0。

```
~/Lab3-AttackLab main ?3 ❯ ./attack/dark-calc < /tmp/n24.txt; echo "exit=$?"
Enter an expression (length up to 15):
zsh: bus error  ./attack/dark-calc < /tmp/n24.txt
exit=135
```

图 5 长度 24 的终端输出。24 个 1 的输出。提示语之后进程以 bus error 终止，退出码 135。

```
~/Lab3-AttackLab main ?3 ❯ gdb -batch -q -ex 'set pagination off' -ex 'set confirm off' \
  -ex 'break *input+94' -ex 'run < /tmp/n24.txt' \
  -ex 'x/32xb $rbp-0x10' -ex 'x/2gx $rbp' ./attack/dark-calc
Breakpoint 1 at 0x4019d5
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
Enter an expression (length up to 15):

Breakpoint 1, 0x00000000004019d5 in input ()
0x721000800fd0: 0x31    0x31    0x31    0x31    0x31    0x31    0x31    0x31
0x721000800fd8: 0x31    0x31    0x31    0x31    0x31    0x31    0x31    0x31
0x721000800fe0: 0x31    0x31    0x31    0x31    0x31    0x31    0x31    0x31
0x721000800fe8: 0x00    0x1a    0x40    0x00    0x00    0x00    0x00    0x00
0x721000800ff0: 0x3131313131313131      0x0000000000401a00
```

图 6 长度 24 的写入后字节。24 个 1 在 leave 前的内存快照。0x...fd0 到 0x...fe7 全为 0x31，0x...fe8 首字节为 0x00；x/2gx 读出保存的 rbp 为 0x3131313131313131、返回地址为 0x401a00。

前两张支撑第 6 节的三块区域定位表与 leave、ret 读数，后三张支撑第 7 节的长度实验表与字节证据表。图中数字与正文表格一致，正文不再重复抄录。

9 困难与易错点

分析层面值得记的有四处，按出现顺序：

一是地址与值的混淆。rbp 那一格的地址是 0x...fe0，格里的内容是 0x...ff0，这两个数只差 0x10，画栈图和填表时容易混。

二是断点位置与寄存器含义的变化。停在 input 入口时 rdi 还是上一条调用的残留值，缓冲区地址要等 +32 的 mov %rax,%rdi 之后才成立；leave 之后 rbp 的含义从本帧基址变回调用者的基址。

三是跨进程的地址。栈地址每次运行不同，先前记下的 0x...ff0 在新进程里不再是同一个值，能跨运行比较的只有代码地址和相对偏移。

四是输出缓冲与观测顺序。重定向到文件时父进程的标准输出是全缓冲，system 子进程的输出先落盘，文件里的顺序与终端不同。

前两条在分析里反复出现。地址与值的区别我一开始混过：rbp 那一格的地址是 0x...fe0，格里的内容是 0x...ff0，把两者写反会让整张对照表失义，后来每填一格都先问一句“这是位置还是内容”。断点位置也一样，停在 input 入口时 rdi 还是上一条调用留下的值，读到 0x1 那种数字就说明停早了；leave 之后 rbp 的含义改变，跨过这条指令再比较 rbp 就失去意义。

第三条影响记录方式。栈地址每次运行都不同，先前抄下的 0x...ff0 在新进程里对应别的字节，所以报告里只写相对偏移与内容，绝对地址留作当次运行的截图说明。

第四条解释了一个反直觉的现象。终端里看到“提示语、Executing、结果”的顺序，重定向到文件后变成“结果、提示语、Executing”，原因是父进程的标准输出经过文件时按全缓冲处理，子进程的输出直接落盘。记录输出顺序时需要说明是哪种运行方式，否则同一组实验会出现两种“事实”。

10 AI 使用说明

本阶段的对话中，AI 承担了三件事：夯实栈帧与寄存器的基础概念、生成注释副本 01_input_annotated.S、把实测记录整理成表格。

下列内容由本人亲手运行并核对，未经核对的推断已在正文标注：序言与 leave、ret 的寄存器读数（见第 6 节前两张表），缓冲区地址与越界字节（见第 6 节区域定位表与第 7 节字节证据表），五种长度的输出与退出码（见第 7 节第一张表），eval 拼接后的 command_buf 内容（0x401a57 断点处）。

leave 的等价展开是先自己写出来的：按 mov %rbp,%rsp 与 pop %rbp 两条逐步算 rsp、rbp 的变化，再拿第 6 节 leave 与 ret 那三行读数核对，三次结果一致。eval 的截断同样先手算：由 len 取 15 推出右括号写在偏移 23，再用 0x401a57 处的 command_buf 内容确认第 16 个字符被覆盖。